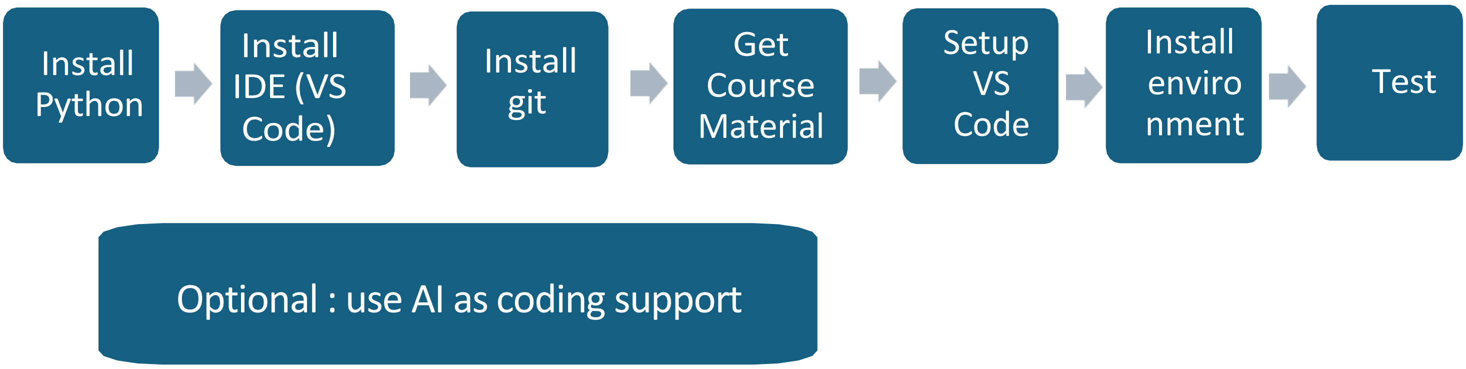
Programming with Python ADVANCED: installation guidelines

The figure below shows the steps for setting up your system so you can work with the coding material and run it on your local system. This preparation might seem like a lot but consider that this is a one-time effort and shouldn't take more than 20 minutes, but, as with most software installations, **it requires you to have administration rights on your computer**. The steps are as follows:



If you experience problems with the installation, for example due to insufficient rights on your computer, skip to the last section "Plan B: working online".

Python Installation

The Python installation step is straightforward and can be accomplished through multiple methods. We recommend downloading Python from the official website at https://www.python.org. Download the latest stable version of Python for your system (Windows, Mac or Linux) from

https://www.python.org/downloads/ .

Download and install the file by following the installation instructions. During installation make sure that you **add the python executable to the PATH environment variable** when asked. For the rest you can stick with the default installation.

IDE Installation

An IDE (Interactive Development Environment) is essential for efficiently writing and managing code in Python projects. Some popular choices for Python include Visual Studio Code, Pycharm and Cursor. I recommend going with Visual Studio Code (or VSCode) for this training. This lightweight and highly customizable IDE supports many different programming languages, which is one reason why it is a favorite. VS Code is open source and free at https://code.visualstudio.com. Its large community provides thousands of extensions. Install it with the download button and follow the instructions. If your are already familiar with another IDE for Python, feel free to continue using it.

Git Installation

Git is an essential tool for version control, allowing developers to track changes in their code, collaborate with others, and maintain a history of project development. Git helps you manage your codebase efficiently by enabling branching, merging, and reverting changes. Git also makes remote collaboration straightforward. You'll need Git in the next step to get the course's material. Download

Git from https://git-scm.com/downloads. Select the download corresponding to your operating system. Many choices are available, but you can go with the default settings during installation.

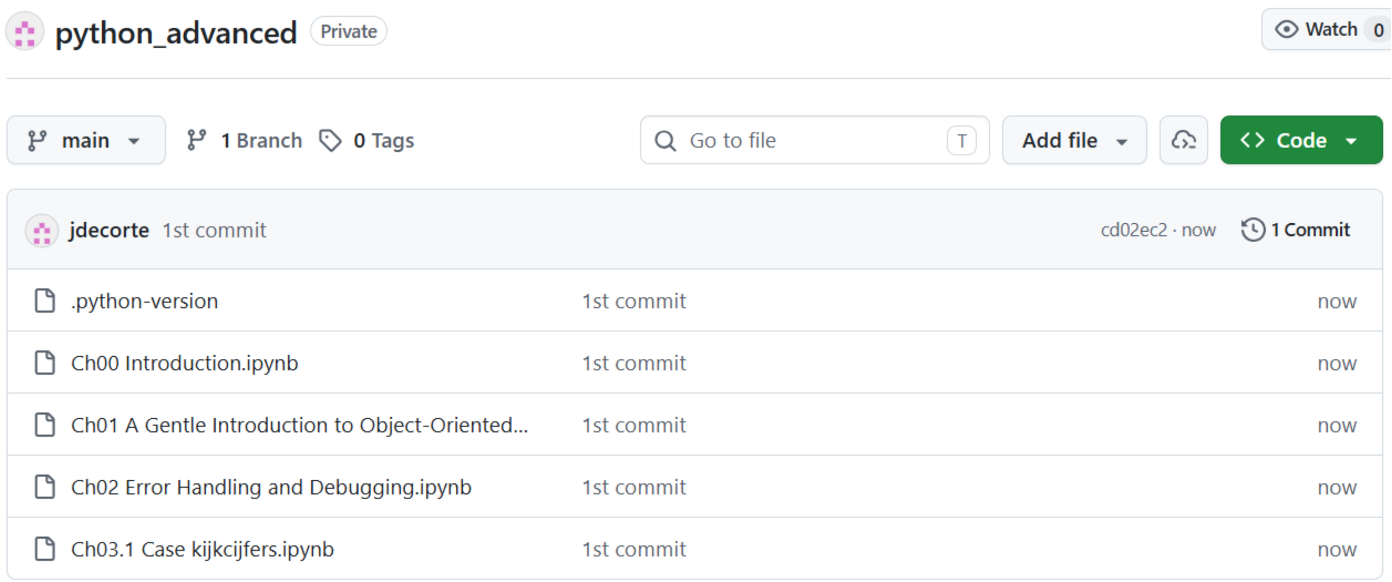
Getting the Course Material

The notebooks (= python code with explanatory text in-between) and images for this course are hosted on GitHub, which is the most prominent website (owned by Microsoft) for sharing and managing program code. You can find the material at the following link:

https://github.com/jdecorte/python_advanced

When you navigate to this page, you should see all the material, as shown in the figure below. You can get the code in two ways by clicking the green Code button.

- Clone using the web URL
- Download as ZIP



If you have no experience with Git, go with the ZIP option. Download the ZIP file and extract it to a folder of your choice on your computer. But if you had points of contact with Git, you could clone the repository by running the following from the command line in a folder of your choice:

```
git clone https://github.com/jdecorte/python_advanced.git
```

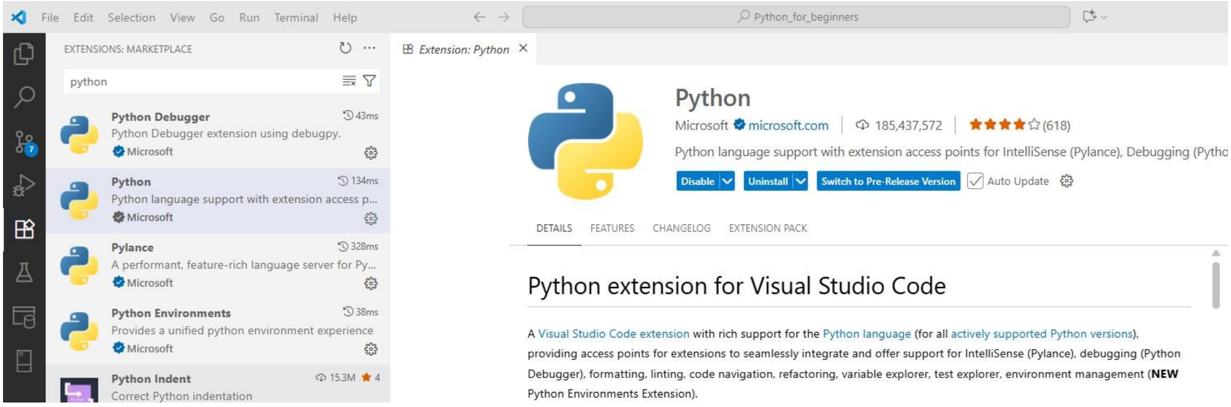
Possibly you will have to switch MS Defender temporarily off.

This command will clone the repository to your computer.

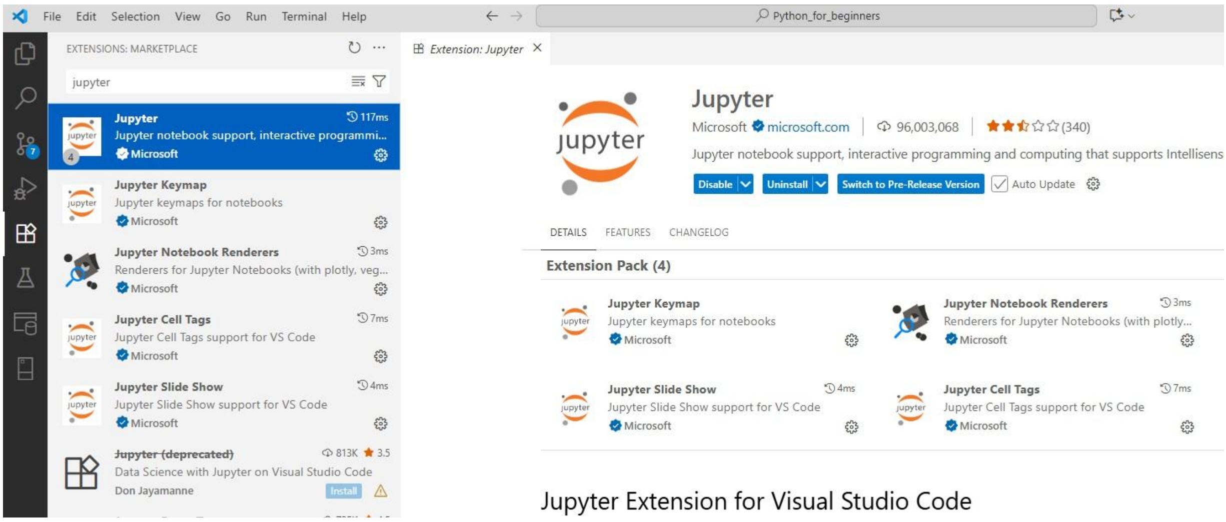
Setting up Visual Studio Code

Visual Studio Code is in fact an empty box. It comes to life when you install extensions. For this course, we need two extensions in VS Code:

- Python: this extension makes we can use the "raw" Python we installed before in an interactive, graphical user interface in Visual Studio Code. Click on the extensions button (this is the 5th button in the vertical toolbar on the left in the screenshot below. Then type "python" in the search field and click the "Python" extension by Microsoft (the second option in the figure). You can now install it.



- Jupyter: with this extension you can read the formatted texts of the course from within Visual Studio Code. Type “jupyter” in the search field and install the extension.



Setting up Your Local Environment

A Python environment is an isolated workspace where you can install and manage dependencies (such as specific versions of Python software libraries you use) for a specific project. This environment acts as a sandbox, ensuring that the libraries and tools used in one project don't interfere with others on your system.

While Project A requires a specific package xyz version 1.0.0 and Python version 3.10, you might also need to work on a different project with completely different requirements. Let's say, for instance, that Project B requires the same package xyz, but in the newer version 2.0.0, and a different Python version, maybe 3.12.7 instead. If you've only set up a single environment for all your projects, you would run into issues because of the incompatibility between the two projects.

The gold standard is to set up an environment for each project you work on. This approach consumes more space on your hard disk, but you'll be on the safe side and avoid negative side effects between your projects. Different tools for managing your Python environments exist but we go for the modern uv.

uv is a less popular but emerging tool, it focuses on creating ultralight virtual environments. Due to its implementation in Rust, it is blazing fast compared to other environmental tools. You can find many different options to install uv on the developer page https://docs.astral.sh/uv/gettingstarted/installation/#standalone-installer. To install and setup uv follow these steps (in a terminal window opened in the folder `python_advanced` where you have put the course material):

- `pip install uv` (or `python -m pip install uv` if that doesn't work) This installs the uv software.
- To fix the Python version in your environment you can use *for example* (only needed if you have multiple Python version installed on your system):

  `uv python pin 3.14.5`
- `uv sync`

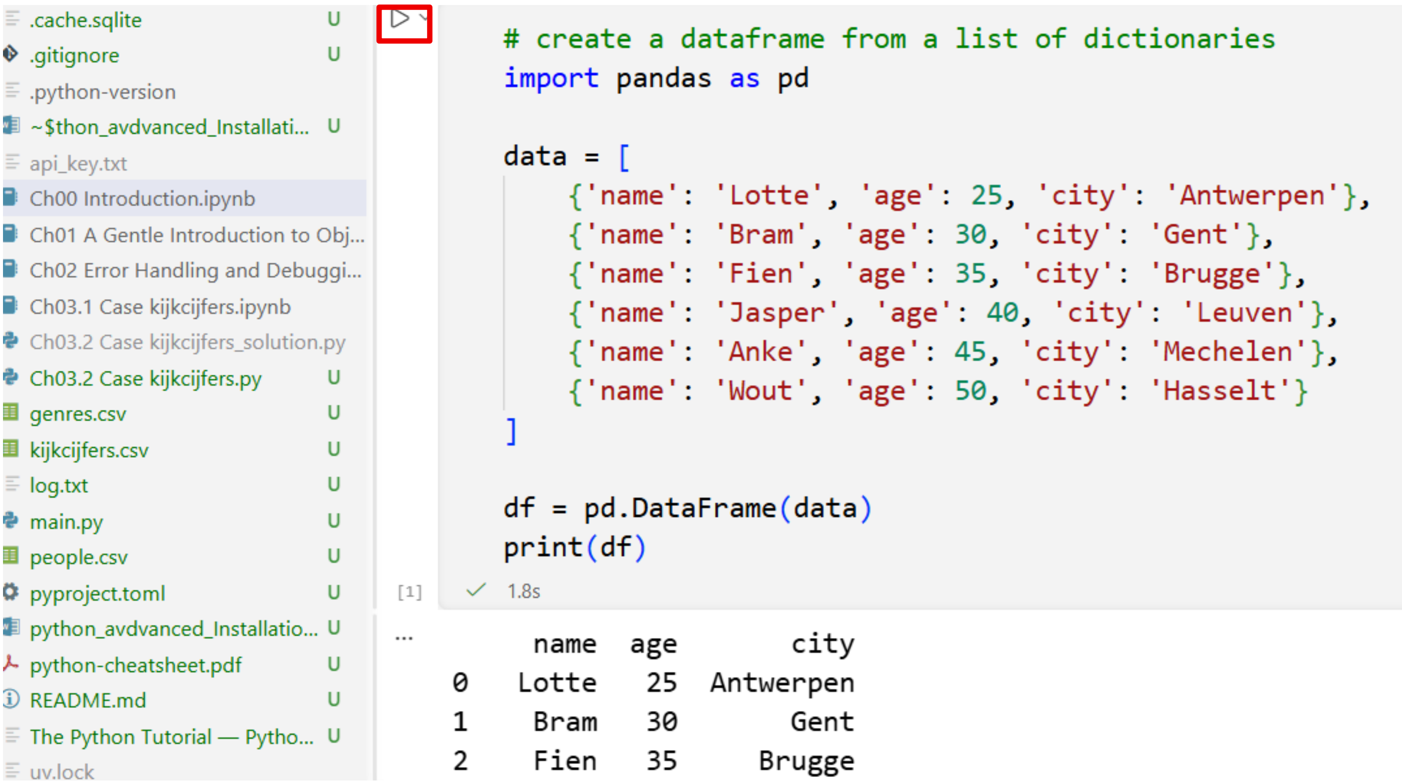
  This command will download and install all dependencies (=libraries used) for this course into a subfolder named `.venv.` It uses a file called `pyproject.toml` in your project folder. This file includes basic meta-data information on the package, as well as package dependencies - the packages and their versions as I have them installed on my system when I created the material.
- If you need to (re)install packages later on you can use

  `uv add <packagename>`

Test your installation

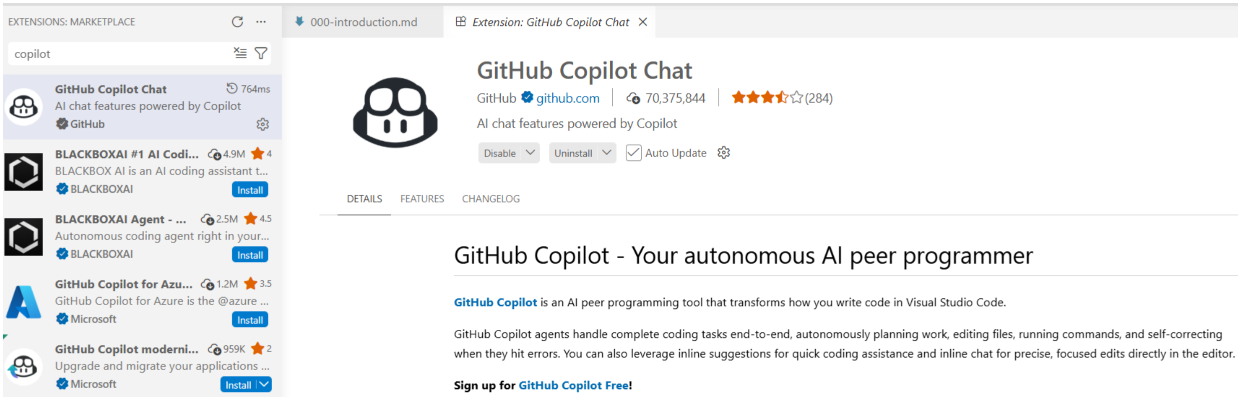
You can now open your `Python_advanced` folder (with File/Open Folder… from the) in Visual Studio Code. Navigate to the file "`Ch00 Introduction.ipynb`" and click the arrow button to the upper left of the first grey cell (accept extra installations if you are asked).

You should get (approximately) the same output as in the figure below (see the terminal pane below for the output).



Optional : use AI as coding support

- Go back to github.com and create a (free) account.
- In VS Code:
    - Install the extension "Github Copilot Chat":



    - Click on **Sign up for GitHub Copilot Free!** in the window above and follow the instructions.

Plan B: working online

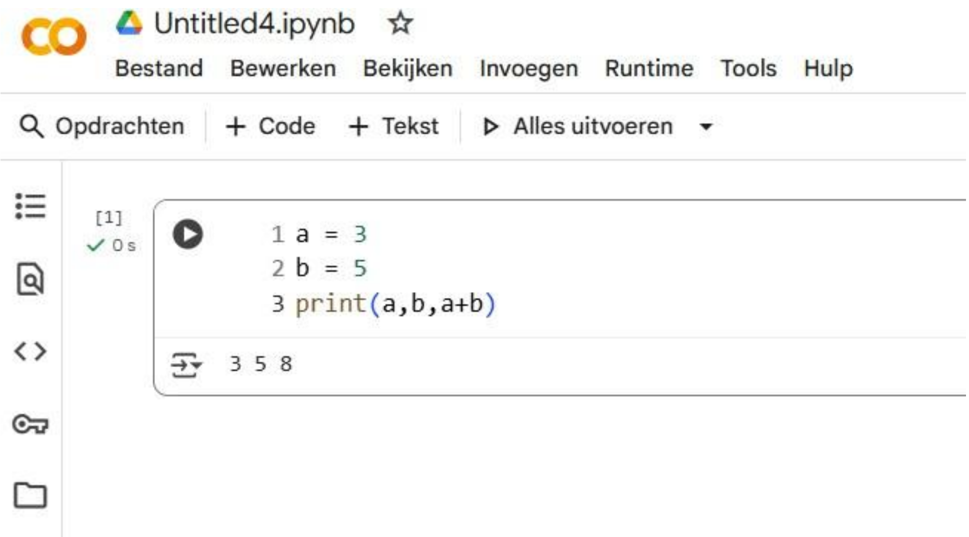
Course Material

You can view the course material at https://github.com/jdecorte/Python_Advanced. Sample code is not executable from there.

Programming in Python

You can write and test your own programs on Google Colab, an online interactive programming environment by Google. Programs are stored online on your google drive, so you need a google account.

Surf to https://colab.research.google.com/ and click "Nieuw notebook". You can now start writing code in the box and click the arrow in front of it to execute:



Keep in mind, this is just a sandbox for playing around with Python. You have no access to your own (local) data sources and you can't deploy real-life programs from there.